ISLS Extension Phase 5 v1.0.0

C21: Adversarial Hypothesis Drill (PMHD)
C22: Universal Artifact IR
C23: Generative Synthesis Engine (FORGE)

The Generative Counterpart:
From Intent to Evidence-Bound Artifact

Absorbing PMHD + ArtifactIR from MERKABA Factory
and Mining + Matrix from OmniCube

Sebastian Klemm

16 March 2026

Abstract

This document specifies the generative half of the ISLS substrate: three crates that transform structured intent into evidence-bound artifacts through adversarial hypothesis drilling, universal intermediate representation, and domain-specific synthesis.

C21 (`isls-pmhd`) implements the Polycentric Multi-Hypothesis Drill: a generative process that produces hypotheses, tests them adversarially against counterexamples, measures quality across six axes (coherence, diversity, novelty, stability, robustness, coverage), and commits survivors as Monoliths.

C22 (`isls-artifact-ir`) defines the universal, flat, serializable intermediate representation that any Monolith is transformed into before domain-specific synthesis.

C23 (`isls-forge`) is the synthesis engine that takes an ArtifactIR, interprets it through a domain Matrix (Filament), evaluates it against constraints, stores patterns in a persistent memory, renders the final output, and validates the result through ISLS's 8-gate cascade — producing a Semantic Crystal that contains the artifact with full evidence provenance from DecisionSpec to final output.

Together with the analytical core (C1–C20), Phase 5 completes the ISLS substrate as a system that can both *discover* programs in data and *generate* artifacts from intent, using

identical formal machinery for both directions.

Status: Implementation Specification.

Parent: ISLS v1.0.0 + Extensions Phase 1–4.

Absorbs: MERKABA `domain-artifact` (PMHD, ArtifactIR, Synth, Eval, Memory, Render), OmniCube `blueprint-forge` (Mining, Matrix, Materializer).

Depends on: C12 (Registry), C13 (Manifest), C14 (Capsule), C17 (Store), C19 (Gateway).

Contents

I Foundations 5

1 The Two Directions 5

2 Absorption Map 6

II Crate C21: `isls-pmhd` 6

3 Purpose 6

4 Core Objects 7

5 Drill Algorithm 8

6 Configuration 8

7 Rust API 9

8 Acceptance Tests 10

III Crate C22: `isls-artifact-ir` 11

9 Purpose 11

10 Data Model 11

11 Acceptance Tests 13

IV Crate C23: `isls-forge` 13

12 Purpose 13

13 Pipeline 13

14 Domain Matrices (Filaments) 14

15 Synthesis 15

16 Evaluation 15

17 Pattern Memory 15

18 8-Gate Crystal Validation 16

19 Emitters 16

20 Full Forge API 17

21 CLI Integration **18**

22 Gateway Integration **18**

23 Acceptance Tests **18**

V The Complete Loop **19**

24 Discovery-Forge Cycle **19**

VI Dependency Graph and Handoff **19**

25 Updated Dependency Graph **20**

26 Test Count **21**

27 Verification Sequence **21**

28 Completion Criteria **21**

Part I

Foundations

1 The Two Directions

ISLS until Phase 4 is purely analytical: it observes data, discovers patterns, and crystallizes them. Phase 5 adds the generative direction: it accepts structured intent, drills it adversarially, synthesizes an artifact, and crystallizes the result through the same 8-gate cascade.

Definition 1.1 (Analytical Direction)**.**

$$\text{World} \xrightarrow{\text{observe}} G \xrightarrow{\text{extract}} P \xrightarrow{\text{validate}} C_{\text{discovery}}$$

Input: observations. Output: crystal encoding a discovered program.

Definition 1.2 (Generative Direction)**.**

$$\text{Intent} \xrightarrow{\text{drill}} M \xrightarrow{\text{IR}} A \xrightarrow{\text{synth}} O \xrightarrow{\text{validate}} C_{\text{forge}}$$

Input: DecisionSpec. Output: crystal encoding a generated artifact.

Axiom 1.1 (Substrate Identity)**.** Both directions use the same state space ($\mathbb{R}^5$), the same canonicalization (JCS + SHA-256), the same Run Descriptor, the same Evidence Chain, the same Crystal format, and the same 8-gate cascade. A crystal produced by the forge is structurally indistinguishable from a crystal produced by discovery. Only the `program_id` field in the manifest distinguishes provenance.

Axiom 1.2 (Adversarial Quality)**.** No artifact may be committed without surviving adversarial opposition. The PMHD drill is not a generator with a quality filter — it is a dialectical process where hypotheses are tested against structured counterevidence before commitment.

Axiom 1.3 (Memory Persistence)**.** The pattern memory is append-only and persistent across runs. Every committed monolith enriches the memory for future runs. The memory is stored in the ISLS archive (C7) and store (C17), not in an ephemeral in-process structure.

2 Absorption Map

Source	Target	Transformation
MERKABA `pmhd/`	C21 `isls-pmhd`	DrillEngine, Hypothesis, Counterexample, QualityMetrics, PmhdMonolith, Provenance. Adapted to use ISLS types, registry, and PoR from C5.
MERKABA `artifact_ir`	C22 `isls-artifact-ir`	ArtifactIR, ArtifactHeader, Component, Interface, Constraint, Metrics, Provenance, Delta. Unchanged structurally; retyped to use Hash256 and ISLS serialization.
MERKABA `synth/eval/memory/render`	C23 `isls-forge`	Synthesizers, Evaluators, PatternLibrary, Emitters. Extended with 8-gate validation and crystal output.
OmniCube `qops`	C21 mining strategies	Greedy, Stochastic, Beam, Evolutionary search strategies for hypothesis seed generation.
OmniCube `blueprint-forge`	C23 domain matrices	Matrix concept: domain-specific interpretation of abstract signatures into concrete artifacts.

Part II

Crate C21: `isls-pmhd`

3 Purpose

The PMHD crate implements the adversarial hypothesis drill: a deterministic process that generates, opposes, evaluates, and commits structured hypotheses about what an artifact should be.

4 Core Objects

Definition 4.1 (DecisionSpec)**.** A DecisionSpec $\mathcal{D}$ is the input to the generative direction:

$$\mathcal{D} = \langle \texttt{id}, \texttt{intent}, \texttt{goals}, \texttt{constraints}, \texttt{seeds}, \texttt{domain}, \texttt{config} \rangle$$

where:

- `id`: content-address of the canonical spec,
- `intent`: natural-language or structured description of the desired artifact,
- `goals`: key-value map of target quality metrics (e.g., {`coherence: 0.7, robustness: 0.8`}),
- `constraints`: hard constraints that must be satisfied (list of predicate strings),
- `seeds`: optional seed crystals from the ISLS archive to guide hypothesis generation,
- `domain`: target domain identifier (e.g., `"rust"`, `"api"`, `"workflow"`),
- `config`: PMHD configuration (tick count, pool size, opposition strength).

Definition 4.2 (Hypothesis)**.** A hypothesis $h \in \mathcal{H}$ is a structured claim:

$$h = \langle \texttt{id}, \texttt{claim}, \texttt{assumptions}, \texttt{evidence}, \texttt{generation} \rangle$$

where:

- `id` = SHA-256(claim ‖ sorted(assumptions)),
- `claim`: the core assertion (what the artifact should be),
- `assumptions`: ordered list of supporting assumptions,
- `evidence`: list of evidence items supporting the claim,
- `generation`: the drill tick at which this hypothesis was created.

Definition 4.3 (Counterexample)**.** A counterexample $c \in \mathcal{C}$ is a structured objection:

$$c = \langle \texttt{id}, \texttt{target_id}, \texttt{reason}, \texttt{severity} \rangle$$

where `severity` $\in [0, 255]$ quantifies the strength of the objection.

Definition 4.4 (Quality Metrics)**.** The quality of a hypothesis is measured across six axes:

$$\mathcal{Q}(h) = \langle q_{\text{coh}}, q_{\text{div}}, q_{\text{nov}}, q_{\text{stab}}, q_{\text{rob}}, q_{\text{cov}} \rangle \in [0, 1]^6$$

- q_{coh}: **Coherence** — internal consistency of the hypothesis and its evidence.
- q_{div}: **Diversity** — breadth of perspectives explored in the drill pool.
- q_{nov}: **Novelty** — distance from previously committed monoliths in the pattern memory.
- q_{stab}: **Stability** — resilience under repeated adversarial opposition across ticks.
- q_{rob}: **Robustness** — survival rate against counterexamples weighted by severity.
- q_{cov}: **Coverage** — fraction of the goal space addressed by the hypothesis.

Definition 4.5 (Monolith)**.** A PmhdMonolith $m \in \mathcal{M}$ is the committed output of a drill cycle that survived all quality thresholds and the PoR gate:

$$m = \langle \texttt{id}, h^*, \texttt{excalibration}, \{c_1, \ldots, c_k\}, \mathcal{Q}(h^*), \texttt{provenance} \rangle$$

where h^* is the surviving core hypothesis, `excalibration` is the adversarial sharpening vector, and provenance contains seed, config hash, tick range, active set stats, and PoR evidence.

5 Drill Algorithm

Algorithm 1 PMHD_DRILL: Adversarial hypothesis drill

Require: DecisionSpec $\mathcal{D}$, Config θ, PatternMemory $\mathcal{P}$
Ensure: List of PmhdMonoliths $\{m_1, \ldots, m_k\}$

```
pool ← SeedHypotheses(D, P, θ)
for t = 1 to θ.ticks do
    // Phase 1: Generate
    pool ← pool ∪ Mutate(pool, D, t)
    // Phase 2: Oppose
    for each h ∈ pool do
        counters ← GenerateCounterexamples(h, D, t)
        h.evidence ← h.evidence ∪ Defend(h, counters)
    end for
    // Phase 3: Evaluate
    for each h ∈ pool do
        Q(h) ← MeasureQuality(h, pool, P)
    end for
    // Phase 4: Select and Prune
    pool ← TournamentSelect(pool, θ.pool_size)
    // Phase 5: Commit Gate
    for each h ∈ pool where Q(h) ≥ θ.thresholds do
        if PoR_Gate(h, Q(h), D) = FIRE then
            m ← CommitMonolith(h, Q(h), t, D)
            monoliths ← monoliths ∪ {m}
            P.add(m)
        end if
    end for
end for
return monoliths
```

Definition 5.1 (Seed Strategies)**.** Hypothesis seeding uses strategies absorbed from OmniCube QOPS:

(i) **Greedy**: seed from the highest-resonance crystals in the pattern memory that match the DecisionSpec domain.
(ii) **Stochastic**: deterministic pseudo-random generation from the RD seed, sampling the 5D signature space.
(iii) **Beam**: maintain k best hypotheses and expand each.
(iv) **Evolutionary**: mutation + crossover on the hypothesis pool with tournament selection.
(v) **Hybrid**: combine greedy seeding with evolutionary expansion (default).

All strategies are deterministic under fixed $(\mathcal{D}, \mathrm{RD})$.

Definition 5.2 (Commit Gate)**.** The PMHD commit gate fires if and only if:

$$G_{\mathsf{PMHD}}(h) = \mathbf{1}[q_{\mathrm{coh}} \geq \theta_{\mathrm{coh}} \ \wedge \ q_{\mathrm{div}} \geq \theta_{\mathrm{div}} \ \wedge \ q_{\mathrm{nov}} \geq \theta_{\mathrm{nov}} \ \wedge \ q_{\mathrm{stab}} \geq \theta_{\mathrm{stab}} \ \wedge \ q_{\mathrm{rob}} \geq \theta_{\mathrm{rob}} \ \wedge \ q_{\mathrm{cov}} \geq \theta_{\mathrm{cov}} \ \wedge \ \mathrm{PoR}(h)$$

The PoR check uses the existing ISLS PoR FSM (C5) applied to the drill state trajectory.

6 Configuration

PMHD Configuration

```yaml
pmhd:
  ticks: 5000
  pool_size: 50
  opposition_strength: 0.7   # severity multiplier
  mutation_rate: 0.3
  crossover_rate: 0.2
  seed_strategy: "hybrid"
  thresholds:
    coherence: 0.6
    diversity: 0.4
    novelty: 0.3
    stability: 0.5
    robustness: 0.6
    coverage: 0.4
  commit_budget: 20           # max monoliths per run
  use_pattern_memory: true
```

7 Rust API

```rust
// isls-pmhd/src/lib.rs

pub struct DecisionSpec {
    pub id: Hash256,
    pub intent: String,
    pub goals: BTreeMap<String, f64>,
    pub constraints: Vec<String>,
    pub seeds: Vec<Hash256>,        // crystal IDs from archive
    pub domain: String,
    pub config: PmhdConfig,
}

pub struct Hypothesis {
    pub id: String,                 // SHA-256(claim || assumptions)
    pub claim: String,
    pub assumptions: Vec<String>,
    pub evidence: Vec<Evidence>,
    pub generation: u64,
}

pub struct Counterexample {
    pub id: String,
    pub target_id: String,
    pub reason: String,
    pub severity: u8,
}

pub struct QualityMetrics {
    pub coherence: f64,
    pub diversity: f64,
    pub novelty: f64,
    pub stability: f64,
    pub robustness: f64,
    pub coverage: f64,
```

```
}

pub struct PmhdMonolith {
    pub id: String,
    pub core_hypothesis: Hypothesis,
    pub excalibration_vector: Vec<f64>,
    pub counterexamples: Vec<Counterexample>,
    pub quality: QualityMetrics,
    pub provenance: MonolithProvenance,
}

pub struct DrillEngine {
    config: PmhdConfig,
    pool: Vec<Hypothesis>,
    monoliths: Vec<PmhdMonolith>,
    pattern_memory: PatternMemory,
    tick: u64,
}

pub enum SeedStrategy {
    Greedy,
    Stochastic,
    Beam { width: usize },
    Evolutionary,
    Hybrid,
}

impl DrillEngine {
    pub fn new(config: PmhdConfig) -> Self;
    pub fn with_memory(config: PmhdConfig, memory: PatternMemory) ->
        Self;
    pub fn drill(&mut self, spec: &DecisionSpec) -> DrillResult;
    pub fn step(&mut self, spec: &DecisionSpec) -> StepResult;
    pub fn monoliths(&self) -> &[PmhdMonolith];
    pub fn pattern_memory(&self) -> &PatternMemory;
}

pub struct DrillResult {
    pub monoliths: Vec<PmhdMonolith>,
    pub ticks_executed: u64,
    pub pool_final_size: usize,
    pub quality_history: Vec<Vec<QualityMetrics>>,
    pub commit_count: usize,
}
```

8 Acceptance Tests

ID	Name	Procedure
AT-P1	Drill determinism	Run drill twice with same spec and RD; verify identical monolith IDs.

AT-P2	Opposition effect	Run with opposition_strength=0 vs. opposition_strength=0.9; verify robustness scores differ.
AT-P3	Commit gate	Set all thresholds to 1.0; verify zero monoliths committed. Set to 0.0; verify all committed.
AT-P4	Quality metrics range	Verify all 6 metrics in $[0, 1]$ for every hypothesis at every tick.
AT-P5	Pattern memory growth	Commit 3 monoliths; verify memory contains 3 patterns. Run again with same spec; verify novelty scores reflect existing patterns.
AT-P6	Seed strategies	Run with each strategy; verify all produce at least 1 monolith. Verify determinism for each.
AT-P7	Hypothesis ID determinism	Create hypothesis with same claim and assumptions; verify identical ID.
AT-P8	Provenance completeness	Verify monolith provenance contains seed, config_hash, tick range, PoR evidence.

Part III
Crate C22: `isls-artifact-ir`

9 Purpose

The ArtifactIR is the universal, flat, serializable intermediate representation that bridges the gap between the abstract monolith (the adversarially-tested hypothesis) and the concrete domain-specific output. It is domain-agnostic: the same IR structure represents a Rust module, an API specification, a workflow definition, a database schema, or any other structured artifact.

10 Data Model

Definition 10.1 (ArtifactIR)**.** An ArtifactIR $\mathcal{A}$ is a flat structure:

$$\mathcal{A} = \langle \texttt{header}, \texttt{components}, \texttt{interfaces}, \texttt{constraints}, \texttt{metrics}, \texttt{provenance}, \texttt{deltas}, \texttt{extra} \rangle$$

Definition 10.2 (ArtifactHeader)**.**

```
pub struct ArtifactHeader {
    pub artifact_id: Hash256,       // content-addressed
    pub version: String,            // semantic version
    pub timestamp_tick: u64,        // drill tick of commitment
    pub layer_index: u32,           // hierarchy depth
    pub por_decision: Option<String>, // PoR evidence JSON
    pub source_monolith_id: String,   // link to PmhdMonolith
    pub domain: String,               // target domain
}
```

Definition 10.3 (Component)**.** A component is an atomic unit of the artifact:

```
pub struct Component {
    pub id: String,
    pub kind: String,       // "function", "type", "endpoint", etc.
    pub name: String,       // human-readable name
    pub content: String,    // canonical JSON of the component body
    pub dependencies: Vec<String>,  // component IDs this depends on
    pub signature: FiveDState,      // 5D embedding of this component
}
```

Definition 10.4 (Interface)**.** An interface defines a contract between components:

```
pub struct Interface {
    pub id: String,
    pub provider: String,    // component ID that provides
    pub consumer: String,    // component ID that consumes
    pub contract: String,    // canonical JSON of the contract
    pub direction: Direction,
}

pub enum Direction { Unidirectional, Bidirectional }
```

Definition 10.5 (ArtifactMetrics)**.** Quality metrics propagated from the monolith plus synthesis metrics:

```
pub struct ArtifactMetrics {
    // From QualityMetrics (monolith)
    pub coherence: f64,
    pub diversity: f64,
    pub novelty: f64,
    pub stability: f64,
    pub robustness: f64,
    pub coverage: f64,
    // Synthesis-specific
    pub component_count: usize,
    pub interface_count: usize,
    pub constraint_satisfaction: f64,  // fraction of constraints met
    pub quality_score: f64,            // composite score
}
```

Definition 10.6 (ArtifactProvenance)**.** Full traceability from DecisionSpec to artifact:

```
pub struct ArtifactProvenance {
    pub decision_spec_id: Hash256,
    pub monolith_id: String,
    pub seed: u64,
    pub config_hash: String,
    pub tick_range: [u64; 2],
    pub drill_strategy: String,
    pub por_evidence: Option<String>,
    pub pattern_memory_size: usize,
}
```

Definition 10.7 (Build from Monolith)**.** The IR is constructed deterministically from a monolith:

```
impl ArtifactIR {
    pub fn build_from_monolith(
        monolith: &PmhdMonolith,
        spec: &DecisionSpec,
        layer: u32,
    ) -> Result<ArtifactIR>;
}
```

The construction is deterministic: same monolith and spec produce identical ArtifactIR (content-addressed).

11 Acceptance Tests

ID	Name	Procedure
AT-IR1	IR determinism	Build IR from same monolith twice; verify identical artifact_id.
AT-IR2	Serde round-trip	Serialize IR to JSON; deserialize; verify equal.
AT-IR3	Provenance link	Build IR; verify provenance.monolith_id matches source monolith.
AT-IR4	Component signature	Build IR; verify every component has a non-zero FiveDState signature.

Part IV
Crate C23: `isls-forge`

12 Purpose

The Forge is the synthesis engine that takes an ArtifactIR and produces a concrete, domain-specific output — validated through ISLS's 8-gate cascade and archived as a Semantic Crystal.

13 Pipeline

Forge Pipeline

```
DecisionSpec
  |
  v
[C21: PMHD Drill] --> PmhdMonolith
  |
  v
[C22: ArtifactIR] --> ArtifactIR
  |
  v
[C23: Matrix Interpret] --> Domain-Specific IR
```

```
  |
  v
[C23: Synthesize] --> Raw Output (code, spec, schema...)
  |
  v
[C23: Evaluate] --> Constraint Check + Quality Bounds
  |
  v
[C23: Pattern Memory] --> Store for future runs
  |
  v
[ISLS: 8-Gate Validate] --> Formal validation
  |
  v
[ISLS: Crystal Commit] --> Semantic Crystal containing the artifact
  |
  v
[C23: Emit] --> Domain-specific files on disk / via Gateway
```

14 Domain Matrices (Filaments)

Definition 14.1 (Matrix)**.** A Matrix $\mathcal{F}$ is a domain-specific interpreter that maps ArtifactIR components to concrete output structures:

```
pub trait Matrix: Send + Sync {
    fn name(&self) -> &str;
    fn domain(&self) -> &str;
    fn interpret(
        &self,
        ir: &ArtifactIR,
        config: &MatrixConfig,
    ) -> Result<SynthesisInput>;
    fn supported_kinds(&self) -> Vec<String>;
}
```

Definition 14.2 (Built-in Matrices)**.** Phase 5 ships with four built-in matrices:

Matrix	Domain	Output
RustModuleMatrix	`rust`	Rust module skeletons with types, traits, impls, tests. Components map to functions/types, interfaces to trait bounds.
HttpApiMatrix	`api`	OpenAPI 3.0 specifications. Components map to endpoints, interfaces to request/response schemas.
WorkflowMatrix	`workflow`	YAML workflow definitions (GitHub Actions / generic). Components map to steps, interfaces to data flows between steps.
SchemaMatrix	`schema`	JSON Schema definitions. Components map to type definitions, interfaces to `$ref` relations.

Custom matrices are registered at runtime via the Matrix trait.

15 Synthesis

Definition 15.1 (Synthesizer)**.** A synthesizer takes the matrix-interpreted structure and produces the raw output string:

```
pub trait Synthesizer: Send + Sync {
    fn synthesize(
        &self,
        input: &SynthesisInput,
        config: &ForgeConfig,
    ) -> Result<SynthesisOutput>;
}

pub struct SynthesisOutput {
    pub content: String,           // the generated artifact text
    pub format: String,            // "rust", "yaml", "json", etc.
    pub components_rendered: usize,
    pub triplet: PerformanceTriplet,
}

pub struct PerformanceTriplet {
    pub psi: f64,    // precision
    pub rho: f64,    // recall / completeness
    pub omega: f64,  // structural quality
}
```

16 Evaluation

Definition 16.1 (Evaluator Stack)**.** The evaluation stack checks the synthesized output against multiple criteria:

(i) **Constraint Evaluator**: verifies all hard constraints from the DecisionSpec are satisfied.
(ii) **Quality Bounds Evaluator**: verifies the PerformanceTriplet meets minimum thresholds.
(iii) **Homeostasis Bridge**: checks that the synthesis did not degrade quality metrics below the monolith's original scores.

```
pub struct EvaluationResult {
    pub constraints_met: usize,
    pub constraints_total: usize,
    pub quality_passed: bool,
    pub homeostasis_passed: bool,
    pub details: Vec<EvalDetail>,
}
```

17 Pattern Memory

Definition 17.1 (PatternMemory)**.** The pattern memory is a persistent, append-only library of committed monolith patterns:

```
pub struct PatternEntry {
    pub monolith_id: String,
    pub domain: String,
    pub quality: QualityMetrics,
    pub signature: FiveDState,   // 5D embedding of the pattern
```

```
    pub component_kinds: Vec<String>,
    pub timestamp: f64,
}

pub struct PatternMemory {
    entries: Vec<PatternEntry>,  // loaded from store at startup
}

impl PatternMemory {
    pub fn load(store: &IslandStore) -> Result<Self>;
    pub fn add(&mut self, entry: PatternEntry, store: &IslandStore) ->
       Result<()>;
    pub fn find_similar(
        &self, signature: &FiveDState, k: usize
    ) -> Vec<&PatternEntry>;
    pub fn novelty_score(&self, candidate: &FiveDState) -> f64;
    pub fn len(&self) -> usize;
}
```

The memory is persisted in `isls-store` (C17) as a dedicated table. Every `add` writes to both the in-memory vector and the database.

18 8-Gate Crystal Validation

Requirement 18.1 (Forge Crystal Validation)**.** Before the forge emits an artifact, the synthesized output **MUST** be packaged as a Semantic Crystal and validated through the standard 8-gate cascade (content_address, dual_consensus, evidence_chain, free_energy, gate_kairos, immutability, operator_versions, por_trace). The crystal's `constraint_program` field contains the ArtifactIR serialized as a canonical JSON blob. The crystal's `topology_signature` encodes the component dependency graph.

Theorem 18.1 (Forge-Discovery Symmetry)**.** *A crystal produced by the forge and a crystal produced by analytical discovery are both valid ISLS crystals that pass the same 8 formal checks. They differ only in provenance: the forge crystal's manifest has* `program_id = "forge:<spec_id>"` *while the discovery crystal has* `program_id = "discovery"`*.*

Proof. By construction: the forge crystal is built using the same `build_crystal_with_id` function, the same evidence chain builder, the same PoR FSM, and the same 8-check validation harness. The crystal format is identical. Only the program_id metadata differs. □

19 Emitters

Definition 19.1 (Emitter)**.** An emitter writes the final artifact to disk, gateway, or external system:

```
pub trait Emitter: Send + Sync {
    fn name(&self) -> &str;
    fn emit(
        &self,
        output: &SynthesisOutput,
        crystal: &SemanticCrystal,
        config: &EmitConfig,
    ) -> Result<EmitResult>;
```

```
}

pub struct EmitResult {
    pub path: Option<String>,
    pub crystal_id: Hash256,
    pub bytes_written: usize,
}
```

Built-in emitters: FileEmitter (write to disk), GatewayEmitter (POST to gateway API), StdoutEmitter (print to console).

20 Full Forge API

```
// isls-forge/src/lib.rs

pub struct ForgeEngine {
    drill: DrillEngine,
    matrices: BTreeMap<String, Box<dyn Matrix>>,
    synthesizers: BTreeMap<String, Box<dyn Synthesizer>>,
    evaluators: Vec<Box<dyn Evaluator>>,
    emitters: Vec<Box<dyn Emitter>>,
    memory: PatternMemory,
    config: ForgeConfig,
}

pub struct ForgeConfig {
    pub pmhd: PmhdConfig,
    pub matrix: String,          // which matrix to use
    pub synth: String,           // which synthesizer
    pub emit: Vec<String>,       // which emitters
    pub validate: bool,          // run 8-gate validation
    pub output_dir: String,
}

pub struct ForgeResult {
    pub drill_result: DrillResult,
    pub artifacts: Vec<ForgeArtifact>,
    pub crystals: Vec<SemanticCrystal>,
    pub manifest: ExecutionManifest,
    pub pattern_count: usize,
}

pub struct ForgeArtifact {
    pub ir: ArtifactIR,
    pub synthesis: SynthesisOutput,
    pub evaluation: EvaluationResult,
    pub crystal_id: Hash256,
    pub emit_results: Vec<EmitResult>,
}

impl ForgeEngine {
    pub fn new(config: ForgeConfig, store: &IslandStore) -> Self;
    pub fn register_matrix(&mut self, m: Box<dyn Matrix>);
    pub fn register_synthesizer(&mut self, s: Box<dyn Synthesizer>);
    pub fn register_emitter(&mut self, e: Box<dyn Emitter>);
```

```
    pub fn forge(&mut self, spec: DecisionSpec) -> Result<ForgeResult>;
    pub fn forge_from_crystal(
        &mut self, crystal: &SemanticCrystal, domain: &str
    ) -> Result<ForgeResult>;
}
```

21 CLI Integration

```
# Forge from a DecisionSpec
isls forge --spec intent.json --domain rust --output artifacts/

# Forge from an existing crystal
isls forge --crystal <crystal_id> --domain api --output artifacts/

# Forge with specific matrix
isls forge --spec intent.json --matrix workflow --output artifacts/

# List available matrices
isls forge --list-matrices

# Show pattern memory stats
isls forge --memory-stats
```

22 Gateway Integration

New Gateway Endpoints (added to C19)

```
POST /forge          Body: {spec, domain, config}
                     -> ForgeResult with crystal IDs and artifacts

POST /forge/crystal  Body: {crystal_id, domain}
                     -> ForgeResult from existing crystal

GET  /forge/matrices List available matrices
GET  /forge/memory   Pattern memory statistics
```

23 Acceptance Tests

ID	Name	Procedure
AT-F1	End-to-end forge	Submit DecisionSpec for Rust module; verify file written and crystal committed.
AT-F2	Forge determinism	Forge same spec twice; verify identical artifact content and crystal ID.
AT-F3	Crystal validation	Forge an artifact; run validate formal; verify 8/8 checks pass.

AT-F4	Forge from crystal	Discover crystal via analytical run; forge it into API spec; verify crystal committed.
AT-F5	Matrix swap	Forge same spec with Rust matrix and API matrix; verify different output formats, both valid crystals.
AT-F6	Constraint rejection	Submit spec with impossible constraint; verify evaluation fails and no crystal committed.
AT-F7	Pattern memory persistence	Forge, restart, forge again; verify novelty score reflects the first run's patterns.
AT-F8	Emitter file	Forge with FileEmitter; verify file exists at expected path.
AT-F9	Emitter gateway	Start gateway; forge with GatewayEmitter; verify crystal queryable via GET /crystals.
AT-F10	Custom matrix	Register custom matrix; forge with it; verify output uses custom interpretation.

Part V

The Complete Loop

24 Discovery-Forge Cycle

With Phase 5, the complete ISLS substrate supports:

1. **Observe**: Data flows in via adapters (C20) through the gateway (C19).
2. **Discover**: The analytical engine (C1–C18) finds patterns and crystallizes them.
3. **Forge**: A discovered crystal is fed into the forge (C21–C23) as a seed, combined with a DecisionSpec, and adversarially drilled into a monolith.
4. **Synthesize**: The monolith becomes an ArtifactIR, interpreted through a domain matrix, synthesized into a concrete artifact.
5. **Validate**: The artifact is packaged as a crystal and validated through the 8-gate cascade.
6. **Emit**: The artifact is written to disk, sent to a webhook, or served via the gateway.
7. **Deploy**: The artifact is used in the world.
8. **Observe**: ISLS observes the effects of the deployed artifact.
9. **Re-discover**: New patterns emerge from the changed world.
10. **Re-forge**: The new patterns seed the next forge cycle.

Every step in this cycle produces crystals. Every crystal is content-addressed, evidence-bound, and replay-verifiable. The entire history is append-only. The system accumulates structural intelligence over time without manual configuration.

Part VI

Dependency Graph and Handoff

25 Updated Dependency Graph

Complete ISLS Dependency Graph after Phase 5

```
isls-types        <-- (all crates)
    ^
isls-registry (C12)
    ^
isls-observe  <-- isls-persist <-- isls-topology (C16)
                        ^                 ^
                        |          isls-extract
                        |                 ^
                        |          isls-scale (C18)
                        |
                isls-consensus <-- isls-carrier
                        ^
                isls-archive <-- isls-morph
                        ^
                isls-manifest (C13)
                        ^
                isls-capsule (C14)
                        ^
                isls-scheduler (C15)
                        ^
                isls-engine
                        ^
                isls-store (C17)
                        ^
                isls-pmhd (C21)         <-- NEW: adversarial drill
                        ^
                isls-artifact-ir (C22)  <-- NEW: universal IR
                        ^
                isls-forge (C23)        <-- NEW: synthesis engine
                        ^
                isls-adapters (C20)
                        ^
                isls-gateway (C19)      <-- extended with /forge
                   endpoints
                        ^
                isls-harness + isls-cli
```

26 Test Count

Phase	New Tests	Cumulative
Core (C1–C9)	130	130
Phase 1 (C10–C15)	31	161
Phase 2 (C16–C17)	20	181
Phase 3 (C18)	15	196
Phase 4 (C19–C20)	23	219
Phase 5 (C21–C23)	22	$\geq$ 241

27 Verification Sequence

```
# Build
cargo build --workspace --release 2>&1 | grep -c warning  -> 0

# Test
cargo test --workspace -- --test-threads=1  -> >= 241 passed

# Forge from DecisionSpec
cat > /tmp/spec.json << 'EOF'
{
  "intent": "Create a REST API health-check module",
  "goals": {"coherence": 0.7, "robustness": 0.8},
  "constraints": ["must return JSON", "must include uptime"],
  "domain": "rust"
}
EOF
cargo run --bin isls --release -- forge --spec /tmp/spec.json \
  --domain rust --output /tmp/forge-out/

# Verify the forged artifact is a valid crystal
cargo run --bin isls --release -- validate formal
# -> Forge crystal passes 8/8 checks

# Forge from a discovered crystal
cargo run --bin isls --release -- ingest --adapter synthetic --scenario
   S-Basic
cargo run --bin isls --release -- run --mode shadow --ticks 100
CRYSTAL_ID=$(cargo run --bin isls --release -- crystal list | head -1)
cargo run --bin isls --release -- forge --crystal $CRYSTAL_ID \
  --domain api --output /tmp/forge-api/
```

28 Completion Criteria

Phase 5 is complete when:

1. PMHD drill produces monoliths deterministically.
2. ArtifactIR builds deterministically from monoliths.
3. Forge produces artifacts through all 4 built-in matrices.
4. Every forged artifact is a valid crystal (8/8 checks).
5. Pattern memory persists across runs.
6. Forge from discovered crystal works end-to-end.

7. Gateway /forge endpoints work.
8. Total tests ≥ 241, zero warnings.
9. **23 Crates. One binary. Two directions. One substrate.**